校园 AI 助手 · 技术文档

传智杯 AI Web 竞赛参赛作品

项目	内容
作品名称	校园 AI 助手（Campus AI Assistant）
参赛单位	阳光学院
团队名称	铁人队
团队成员	见提交材料
指导教师	见提交材料
联系方式	见提交材料
构建号	2026-11-01-a
在线演示	https://campus-ai-assistant-61569.app.workbuddy.host/
评审账号	学生 `demo` / `demo123456`；管理员 `admin` / `zrc666`
代码仓库	https://github.com/1280507078/campus-ai

一、项目概述

1.1 作品定位

校园 AI 助手是一个面向在校大学生的 Web 应用，把四件最常发生、却分散在四五个系统里的事收进一个浏览器页面：

- **学习规划**：按周次显示的时间表、Excel / CSV 课表导入（带解析预览），以及能调用工具查真实数据的 AI 排计划；
- **学校事务问答**：把教务处公开的 29 篇办事指南与校历做成可检索知识库，回答"休学要准备什么材料""校园卡丢了怎么补"，并**标出出处**；
- **情绪陪伴**：低压力的对话入口，带长期记忆；8 题自评量表，出分与分级建议；
- **天气地图导航**：高德底图 + **自建校园步道路网**，给出"从这个楼走到那个楼到底多少米"的校内路线，并附当日天气提示。

外加一套给学校管理员用的**数据看板与反馈处理**页面。

它不是一个"聊天框套壳"。所有需要精确的环节（第几周、哪一节课、多少米、几分钟、几分）都由代码计算，大模型只负责理解意图与组织语言——这是贯穿全篇的一条设计红线。

1.2 交付形态

维度	选择	理由
部署单元	单端口（后端同时托管前端静态文件）	评审不需要开两个服务、不需要配跨域
前端	原生多页面（HTML/CSS/JS），**零构建**	不需要 Node.js、不需要 `npm install`，双击即用
数据库	SQLite（可切 MySQL）	零安装；单文件可迁移
启动	`start-windows.bat` 双击 / `Dockerfile`	本地一键；服务器一条 `docker build`
外部服务	GLM / 高德 / Qdrant，**全部可缺**	缺任何一个都不白屏，只降级对应功能

作品长什么样（本地真实运行截图，`demo` 账号，浅色主题）：



图 1　首页：三张功能卡 + 今日一览（本周 7 门课、校内 24 个点位、今日 0 门课）

首屏刻意**不放聊天框**：先给"今天上什么课、现在几度、有什么要办"，聊天入口放在「学习规划」里。理由是问卷里 94.3% 的人每天第一个问题是"几点上课、在哪栋楼"——这件事不需要模型，也不需要打字。

1.3 规模与质量

指标	数值
后端 Python	10,672 行（FastAPI + SQLAlchemy）
前端 JS	9,564 行
前端 CSS	3,055 行（一套设计令牌，浅色 / 黑夜 / 护眼三主题）
测试与工具脚本	8,317 行 / 18 个 `test_*` 脚本
页面	14 个（多页面，非单页路由）
后端接口组	6 组（auth / planner / emotion / map / feedback / admin）
AI 工具（function calling）	6 个，全部只读
知识库	29 篇校内文档 → 向量检索（400 字/块，60 字重叠）
自建校园路网	172 个节点 / 205 条路段 / 590 个折线中间点
时间槽	18 个（06:00–23:00，按学校真实节次）
测评量表	8 题（4 项抑郁 + 4 项焦虑，4 级计分）

1.4 文档结构

第二章说明需求从哪来；第三章给出架构与选型；**第四章是 AI 技术方案（选型理由、集成方式、优化策略）**；第五章讲核心功能怎么实现；第六章讲测试与部署；第七章总结；**第八章单列创新点。**附录给出环境配置、依赖清单与评审快速指引。

二、需求分析

2.1 调研方法与样本

2026 年 10 月 17—25 日，面向本校同学发放匿名问卷，不收集姓名与账号，回收 **53 份**有效样本（大三占 77.4%，其余为大一大四及其他年级）。问卷只问行为，不问态度，避免"你觉得需不需要"这类引导性问题。

关键结果：

问题	结果
出门前会确认"今天几点上课、在哪栋楼"	**94.3%**
还会估算"从宿舍走过去大概多久"	**81.1%**
会看"今天会不会下雨、要不要带伞"	**71.7%**
上述三项的人均勾选数	**2.6 项**
两栋楼之间换课时"时间算错、差点迟到或已迟到"	**49.1%**
其中"几乎每周都有"	**15.1%**

这组数字直接决定了首页放什么：**不是先做一个聊天框**，而是把"课表 + 走过去多久 + 天气"放在首屏，让 AI 去补"页面上查不到"的那部分。

2.2 办事流程：一个被低估的需求

问卷之外，教务处官网上与学生相关的页面被逐一清点：**29 篇**办事指南与校历文件。它们有一个共同特点——**写得都对，但没人愿意在看之前先找。**

这类需求与"查课表"本质不同：

	查课表	问办事流程
答案在哪	数据库里，结构固定	一篇自然语言文档里
出错代价	看错一节课	按编造的流程跑一趟，白跑
正确的"不知道"	少见	**常见**（29 篇覆盖不到的问题远比覆盖到的多）

所以这一块的目标不是"答得多"，而是**答得准，不该答的时候明确说没写。**

2.3 需求归纳

需求	现象	现有工具为什么不够	本项目的应对
时间表要"按周看"	课表不是每天都一样，单双周、前后八周差别很大	教务系统导出的是整学期大表，手机上看不清	周次模型（锚点 + 偏移），按周显示、按周判断
路线要"走校内"	公共地图按机动车路网算，校内小路不认	知行楼→平远楼：高德 1,675 m（绕出校门），实际不到 500 m	自建步道图 + Dijkstra
问句要"答得上"	问课、问楼、问天气要来回切应用	通用助手不知道你的课表	工具调用：先查真实数据，再组织语言
流程要"查得到出处"	办事指南散落在官网附件里	通用 AI 会编，而且编得像	29 篇语料 + 三道反幻觉闸 + 标注出处
情绪要"有出口"	心情差时不想跟熟人讲，也不想填长问卷	心理中心有门槛；通用 AI 无记忆、无分级	低压力对话 + 长期记忆 + 自评量表

2.4 非功能需求

这几条不是"顺带做到"，而是**写进验收标准**的：

1. **零 Key 可运行**：评审不需要申请任何 API Key，就能打开全部页面、看到全部界面。缺 Key 只降级对应功能，不报错、不白屏。
2. **零构建**：不要求评审装 Node.js。前端是纯静态文件，后端一开即可用。
3. **隐私最小化**：问卷匿名；对话内容只用于本人回看；`campus_ai.db` 与导出数据不进仓库、不进交付包。
4. **移动端可用**：320 / 375 / 390 三档实测，顶栏、表格、地图不横向溢出。
5. **可解释**：AI 的每一次工具调用都渲染成时间线，手册问答的每条答案都带原文出处。

三、系统架构设计

3.1 总体架构

系统分四层，层间只通过明确定义的接口通信：



图 1 系统总体架构：四层单向依赖，外部服务全部可缺（缺则降级对应功能）

图 2 系统总体架构

为什么是"多页面"而不是单页应用：这个项目有 14 个页面、共享一套 CSS 与 8 个公共脚本。用单页框架需要引入构建链（Node + 打包器 + 路由库），而收益只是"切页不闪白"。在"评审必须能一键跑起来"这个约束下，**零构建的价值高于单页的顺滑。**跨页体验用 View Transition（Chrome 126+ 自动生效，其余浏览器忽略）补上，不引入依赖。

3.2 技术选型与理由

选择	备选	为什么选它
FastAPI	Flask / Django	原生 async（模型调用是长 IO，同步框架会堵住整个进程）；Pydantic 自动校验请求体；`uvicorn` 单命令起服务
SQLite	MySQL / PostgreSQL	单文件、零安装、可直接拷走；并发量级（教学演示）完全够用，保留 `DATABASE_URL` 切换 MySQL
原生多页面	Vue / React	见 3.1；评审打开源码就能读懂，不需要先理解框架约定
GLM-4-flash	GPT / Claude / 本地模型	官方承诺**永久免费、不限量**，评审可零成本复现；中文理解与工具调用够用；OpenAI 兼容，换模型只改一行 `.env`
手写 RAG	LangChain / LlamaIndex	全流程只有"切块 → 向量化 → 检索 → 拼接"四步，框架带来的抽象成本高于收益；且本项目要在检索后插入三道**自定义的反幻觉闸**，框架反而挡在中间
Qdrant	FAISS / pgvector	托管有免费额度，也可本机 Docker；payload 过滤比 FAISS 方便；缺失时只降级"情绪记忆"与"手册问答"两个功能
高德 JS/Web 服务	百度 / 腾讯地图	校园矢量底图可截取（自建路网的几何来源）；JS API 的步行规划可作对照基准

3.3 数据模型

表	作用	关键约束
`users`	账号、昵称、密码哈希、是否管理员	`username` 唯一
`time_slots`	18 个时间槽（06:00–23:00）	南北区作息各一套
`courses`	课程：名称、教师、楼、教室、星期、节次、周次	`uq_user_date_slot`（同一用户同一格不重复）
`chat_sessions`	情绪对话记录（一问一答一条）	只存当轮，不存整段
`assessments`	测评结果（分数、分级、时间）	分数由后端计算
`feedback`	用户反馈（类型、内容、处理状态）	管理端按状态筛选

课程表**不存"某天的课"**，只存"星期几 + 哪几周"——这是整个时间表模块的地基（见 5.1）。

3.4 一次 AI 请求的链路

以"帮我排一份这周的复习计划"为例：

1. 前端 `plan.html` 收集目标文本、天数、当前课表摘要，POST `/api/planner/plan`；
2. 后端 `routers/planner.py` 校验登录态，交给 `study_agent` 分派；

3. **路由三层判据**先决定走哪条路：学习教练 / 文档问答 / 规划 Agent（见 4.5）；
4. 规划 Agent 构造系统提示（含**今天的日期**、用户课表、6 个只读工具），进入多轮循环；
5. 模型决定调用 `get_timetable` → 拿到真实课程 → 再调 `find_free_slots` 算空档 → 生成计划；
6. 每一轮的工具名、参数、返回摘要写进 `trace`，随结果一起返回；
7. 前端把 `trace` 渲染成时间线，计划正文逐字显示；
8. 任何一步失败（超时/额度/解析失败）→ 降级到**本地规则计划**（毫秒级、不联网），页面不空。

四、AI 技术方案

4.1 AI 能力全景

本项目用了四处 AI，边界划得很清楚：

能力	模型	输入	输出	兜底
学习规划 Agent	GLM-4-flash（工具调用）	目标 + 课表 + 日期	计划文本 + 工具轨迹	本地规则计划
学校文档问答	GLM-4-flash + embedding	问题 + 检索片段	答案 + 出处	明确回答"手册没写"
情绪陪伴	GLM-4-flash + embedding	消息 + 相关记忆	回复	固定关怀话术
天气提示	GLM-4-flash（可选）	天气 JSON	一句出行建议	直接展示温度/降雨原始值

AI 不该做的事同样明确：算第几周、算空档、算里程、算分数、判断"片段里有没有这个动作"——全部由代码算（见 4.5 第 1 条）。

4.2 选型理由

为什么是 GLM-4-flash

- **成本**：官方承诺永久免费、不限量。评审复现不需要充钱——一个需要付费 Key 才能演示的作品，在评审环节等于不可用。
- **能力匹配**：模型任务只有三类：理解中文意图、选择工具、组织一段通顺的话。不需要长链推理，不需要 128k 上下文。
- **可替换**：走 OpenAI 兼容协议，`.env` 里改 `GLM_MODEL` 就能换 `glm-4-plus` 或其它兼容服务，代码零改动。

为什么用 function calling 而不是把数据塞进提示词

把课表直接塞进提示词、让模型"看着课表回答"会导致两类错误：

1. **数错**：把 5 条课程记录当成"每天 5 门课"；
2. **编造**：课表里没有周六的课，它按"大学一般周六有课"补了一节。

改成工具调用后，模型只能拿到工具返回的真实数据，**没有数据就没法编**。这也是"过程可见"的前提：轨迹里没有 `get_timetable`，答案里就不该出现具体课程。

为什么 RAG 不用框架

流程只有四步，但每一步都要插手：

- 切块要按"办事流程的段落感"切（400 字/块），而不是按固定字符数硬切；
- 检索后要判断**分数是否够**、**动作词是否对得上**、**对象是不是同一件事**——这三条判据是本项目的核心资产，放在框架的 callback 里只会更难读；
- 答案必须带 `source`（原文链接），这是"不编"的可验证前提。

自己写大约 300 行，换来的是**每一行都能解释给评委听**。

4.3 集成方式



图 2　一次 AI 请求的处理链路与降级路径

图 3　一次 AI 请求的处理链路

六个工具都是**只读**的，没有"写"操作：

工具	作用	关键参数
`get_timetable`	取整学期课程	无
`get_class_schedule`	取某天/某周的课	`date` / `week`
`find_free_slots`	算某区间的空档	`start_date`、`end_date`、`min_minutes`
`get_building_info`	楼栋位置与用途	`building`
`get_weather`	当日天气	无
`search_campus_docs`	检索校内文档（返回片段 + 出处）	`query`、`top_k`

上下文怎么构造：系统提示里固定注入"今天的日期"与"本周是第几周"。不注入的话，模型会按训练数据里的时间去理解"明天""下周"，实测能把日期算错一整年。

结构化输出怎么保证：`llm_json.py` 负责三件事——剥掉 Markdown 代码围栏、去掉尾随逗号、截取最外层 `{}` / `[]`。模型偶尔会在数组末尾多写一个逗号，`json.loads` 直接抛错；外层若只看异常，就会把**内容**

完全正确的结果白白丢掉——这三步清理比"失败重试一次"便宜得多。

4.4 提示工程与硬约束

三条约束写在系统提示与代码里，缺一不可：

1. **只能引用工具返回的数据**：课表里没有的课不许出现；工具没调用成功时，回复必须是"没查到"，不能给建议式编造。
2. **日期锚点**：所有相对时间（今天/明天/这周/下周）都换算成绝对日期再交给模型，模型只做语言组织。
3. **不确定就说不确定**：导入解析存疑的行在预览页标出来让人确认，**不猜**；手册问答检索为空或动作词缺失，直接答"没写这条"。

4.5 优化策略

#	策略	做法	效果
1	**能算的不调模型**	第几周、空档、里程、分数、"有几个楼"全在本地算	毫秒级返回，且不会算错
2	**路由三段判据**	学习教练 / 文档问答 / 规划，先分流再进模型	避免"问休学答成一张作息表"
3	**只读工具 + 白名单**	6 个工具全部只读，模型无法写库	排计划不会污染真实课表
4	**预算控制**	整轮 Agent 有总预算（`BUDGET_S`），剩余不足直接降级	避免网关 504，用户最多等一个预算周期
5	**超时按输出长度定**	实测 7 天计划 **69.6 秒**（约 1900 字）→ 超时设 180 秒	原先 60 秒超时导致"每次都降级成占位计划"
6	**导入先预览再落库**	解析结果进预览页，逐行确认后才写库	错一行不至于污染整学期课表
7	**JSON 容错**	剥围栏 / 去尾随逗号 / 截取最外层	把"内容对但格式错"的结果救回来
8	**检索为空不调模型**	命中分数不够 → 直接答"手册没写"	省一次调用，且不给"编"的机会
9	**懒加载可选依赖**	`qdrant-client` 惰性导入	没装向量库也能启动，只少两个功能

第 5 条需要展开：**超时的瓶颈不是网速，是输出长度。**大模型逐字生成，7 天计划约 1900 字，实测 69.6 秒；天数调到 14 天就要 140 秒以上。把超时从 60 秒改到 180 秒之前，日志里刷的全是"GLM 请求超时（60s）"，用户看到的是"生成计划用不了"——真正的原因只是**等待时间不够。**

4.6 可靠性工程

降级矩阵（缺任何一项都不白屏）：

缺失	表现	兜底
`GLM_API_KEY`	排计划 / 对话 / 文档问答不可用	明确提示"去 backend/.env 填 Key"，本地规则计划照常出
`QDRANT_URL`	情绪记忆与文档检索失效	对话本身不受影响；文档问答转为"手册没查到"
高德 Key	地图底图与天气不可用	页面显示说明，课表/计划/测评照常
模型超时 / 额度用尽	本轮回答失败	返回本地规则计划，页面不空

降级不是"catch 住不报错"：每一条降级都要说清"为什么降级、怎么办"。所有异常都被吞掉并打日志，但上层拿到的是空串，由调用方转成**用户能看懂的一句话**，而不是一个空白气泡。

五、核心功能实现

5.1 时间表

周次模型（整个模块的地基）

课程只存"星期几 + 第几周"，不存具体日期：

- 锚点 = 开学日（写死在配置里，换学期只改这一个常量）；
- `weekOffset` 表示"相对第 1 周的偏移"，不是"相对本周"；
- 判断某天有没有某门课 = **星期相同** 且 **该天的周次落在课程的 weeks 区间内。**

早期 `weekOffset` 是"相对本周"的，于是"第 1 周"**不是一个固定日期**——今天打开是 8/25，下周打开变成 9/1，一直在漂。而学校说的"第几周"是绝对概念（教学进度、考试周次都按它），会漂就没法用。

为什么不能写 `course.date === 今天`：课程按周重复，`date` 只说明"星期几"。曾有一版拿 `date` 当锚点做区间过滤，结果"这周有课吗"永远返回 0 条。现在前后端各有一份同口径实现，并由守卫测试钉住两边结果一致。

Excel 导入的真实世界宽容度

导入器针对的不是规范表，是学校导出的脏数据：

脏法	处理
四种表头写法	全部识别
四种星期写法（周一/星期一/礼拜一/数字 1）	全部识别
两种连堂排版（一行两节 / 两行各一节）	归并成同一门课
楼名口语简称（"启智 A301"）	补全为"启智楼 A301"
一格认不出	**标黄让人过目**，不静默丢弃、不整表报错

原则一句话：**一格脏数据最多让那一行标黄，绝不让整张表报废。**

解析器**不依赖 pandas**：装了就用，没装就走 `xlrd` / `openpyxl` 直读，两条路输出逐字节相同（守卫测试把 pandas 屏蔽后跑一遍做断言）。这来自一次真实事故——`requirements.txt` 漏了 pandas，而导入器无条件 `import pandas`，于是缺库的机器每次导入都 `ModuleNotFoundError`，重装也没用。

同格冲突：同一用户、同一格（`uq_user_date_slot`）不能有两条课。加课时若与已有课冲突，先问"是否本周也上"，同意则并入已有课程的 weeks，而不是新建一条撞唯一索引。

5.2 AI 伴学 Agent

三条路由（`study_agent` 的入口分派）：

路由	判据	处理
学习教练	`_is_explain`：求方法而非求数据（"怎么复习高数"）	不调工具，直接给方法
文档问答	`_is_docs_question`：强特征词 **或** 疑问形态 + 事务对象	检索 → 三道闸 → 作答
规划 Agent	其余	6 工具多轮循环 → 计划

这条分流的由来：本树原先只有两条路，其余全归规划 Agent，于是「休学需要准备什么材料」被当成排计划，返回的是「08:00-10:00 学习专业课内容」——答得一本正经，但完全答非所问。规划 Agent 的提示词整篇都在教它"怎么排表"，指望它在排表人格里主动去查手册，做不到。

循环：系统提示 → 模型 → 工具 → 回灌 → …（最多 N 轮）→ 正文。每一轮的 `tool_calls` 与返回值摘要都记进 `trace`。

过程可见：前端把 `trace` 渲染成时间线——"读时间表 → 算空档 → 查天气 → 出计划"。这带来一个有益副作用：一旦轨迹暴露在外，就再也没法用"模型自己发挥"敷衍过去，因为**轨迹里没有的调用，答案里就不该出现。**

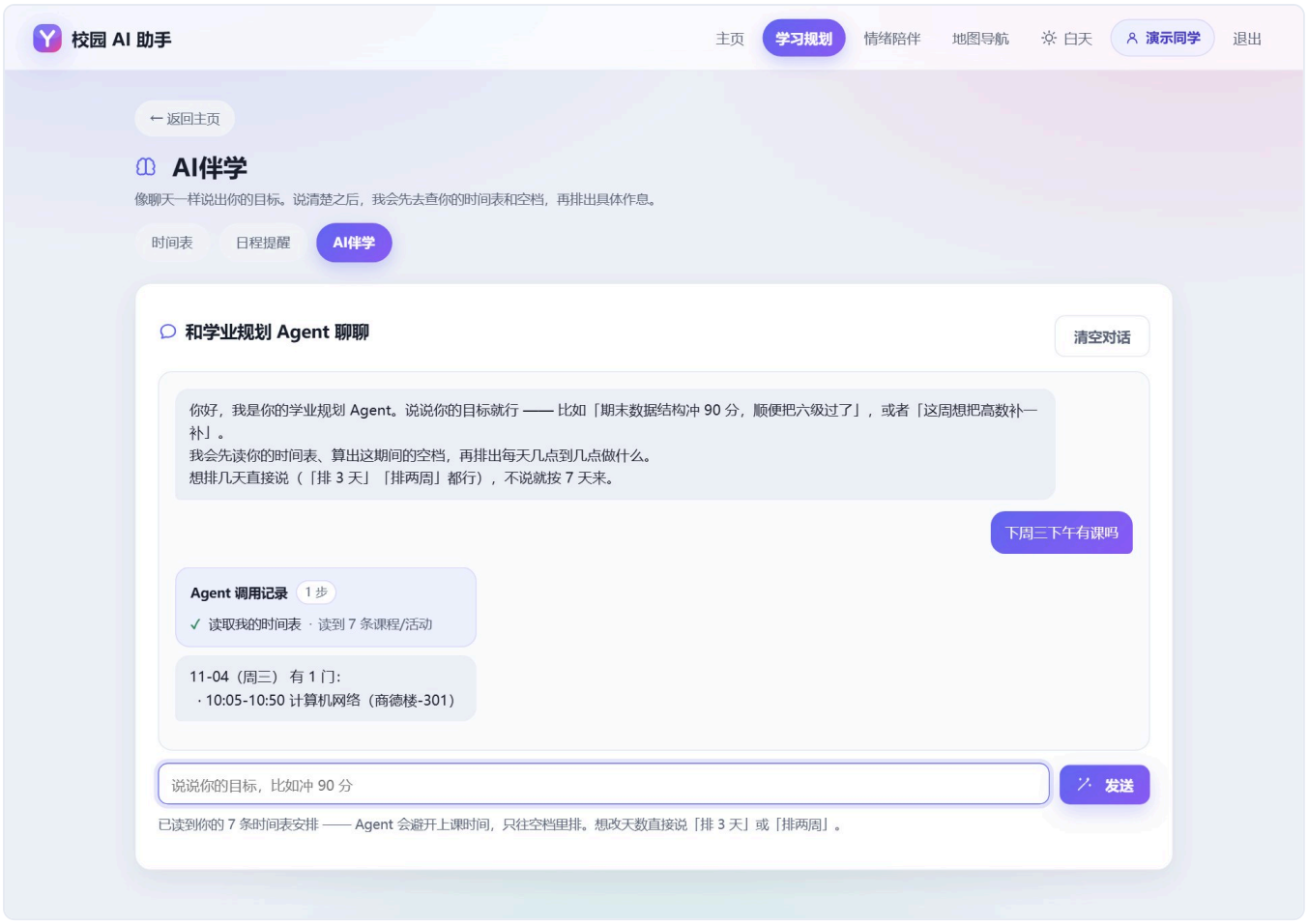


图 4　AI 伴学：先亮出「Agent 调用记录 · 读取我的时间表 · 读到 7 条课程」，再给结论

上图问的是"下周三下午有课吗"：Agent 先去读时间表（调用记录 1 步），再答出「11-04（周三）10:05-10:50 计算机网络（商德楼-301）」。而这类"能直接算"的问题在正式分派里其实走**本地规则**、不调模型——截图里保留调用记录，是为了让过程本身可核对。

本地规则计划：一份不联网、毫秒级返回的排表逻辑。它不是"备胎凑数"，而是保证"AI 挂了页面也有东西看"的底线。触发条件：无 Key、超时、预算耗尽、JSON 解析失败、工具连续失败。

5.3 手册问答：把"不知道"变成可信的答案

语料：教务处官网公开的 29 篇办事指南与校历，每篇 Markdown 开头带原文链接与抓取日期；切块 400 字、重叠 60 字，写入 Qdrant `campus_docs`（与情绪记忆分属两个集合）。

三道闸（按实际执行顺序）：

1. **检索分数不够 → 不调模型**

命中分数低于阈值时直接回答"手册里没查到这条"，一次模型调用都不发。既省成本，也不给"编"的机会。

1. **动作词缺失 → 硬拦**

问句里的动作（补办/挂失/报修/打印……）在检索片段里**一个字都没出现**，就明确说"手册没写这条流程"，不调模型。

1. **对象张冠李戴 → 硬拦**

问「成绩怎么查」却命中《学生查卷申请办理流程》，两者都含"成绩"但办的是完全不同的两件事——判为不匹配并说明"检索到的是另一件事"。

第 2 条需要展开：

> 语料从 14 篇扩到 29 篇后，问「校园卡丢了怎么补」，检索命中了《校园一卡通线上充值指南》——两篇都有"校园卡"，余弦相似度够高。模型照着这份**充值**指南编了一套**补办**流程，还标了个看起来可信的出处。三句话里没有一句是真的。

光靠提示词"片段里没有的不要写"拦不住：片段里确实有"校园卡"，模型判断不了"有校园卡 ≠ 有补办"。所以把**哪个动作词没出现算出来直接摆给它看**——这是确定性的，不依赖模型自觉。同一族词只能有一个来源（`_DOC_ACTION` 一张表），否则"请**病**假"会被另一条正则判成"问病假"，绕过去答了退学。

出处：每条答案都从检索片段带上 `source`，前端渲染成可点链接。这让"不编"变成**可验证**的：用户点开链接就能核对原文。

5.4 校园路网导航

问题：公共地图把校园当机动车路网。知行楼 → 平远楼，高德给 **1,675 米**（绕出校门），两栋楼之间实际不到 500 米。

做法（全流程自研，脚本可复现）：

```
高德矢量底图（z19 / dpr2 网格截取，关掉注记文字）
  → 纯白分割（只取"地图上真画出来的白色道路"）
  → 骨架化 → 矢量化 → 抽稀
  → T 型接点插入 → 端点 2m 聚类 → 建边 → 跨隧道补桥
  → 172 节点 / 205 路段 / 590 折线点
  → Dijkstra 求最短路
```

为什么只用白色道路：判断路线好不好，唯一有意义的标准是"它是不是压在地图上那条白路上"。翻车过两次，原因都是几何来源不干净：

- **OpenStreetMap 的 footway/path/steps**：实测 **80% 的路段离底图路面超过 4 米**，画出来就是"不在路上"；
- **高德步行规划密集探测**（234 条折线 / 112.7 公里）：它自己很准（离白路 p90 = 0.00 米），但其中 6.5% 是"高德会走、地图不画"的未绘制小径，只用来补栅格没提取到的残段，最终净贡献 **0.07 公里**。

收益：

路线	公共地图	本项目
知行楼 → 平远楼	1,675 m（绕出校门）	**680 m**
知行楼 → 致远楼	1,767 m	**519 m**
知行楼 → 五期餐厅	1,839 m（绕出校门）	**520 m**

质量可核验：把路线投到底图真实道路像素上量——节点离路距离 p90 = **3.27 米**；采样点离路 p50/p75/p90 **全部为 0**；离路超过 3 米的里程占比仅 **5.7%**。

坐标系：浏览器给 WGS-84，高德底图是 GCJ-02，福州一带差约 **500 米**。但微信/QQ 内置浏览器本来就返回 GCJ-02，无条件转换会再偏 500 米。做法是**两个候选都算出来，谁离当前路线近就用谁**，并锁定结论防止来回闪。

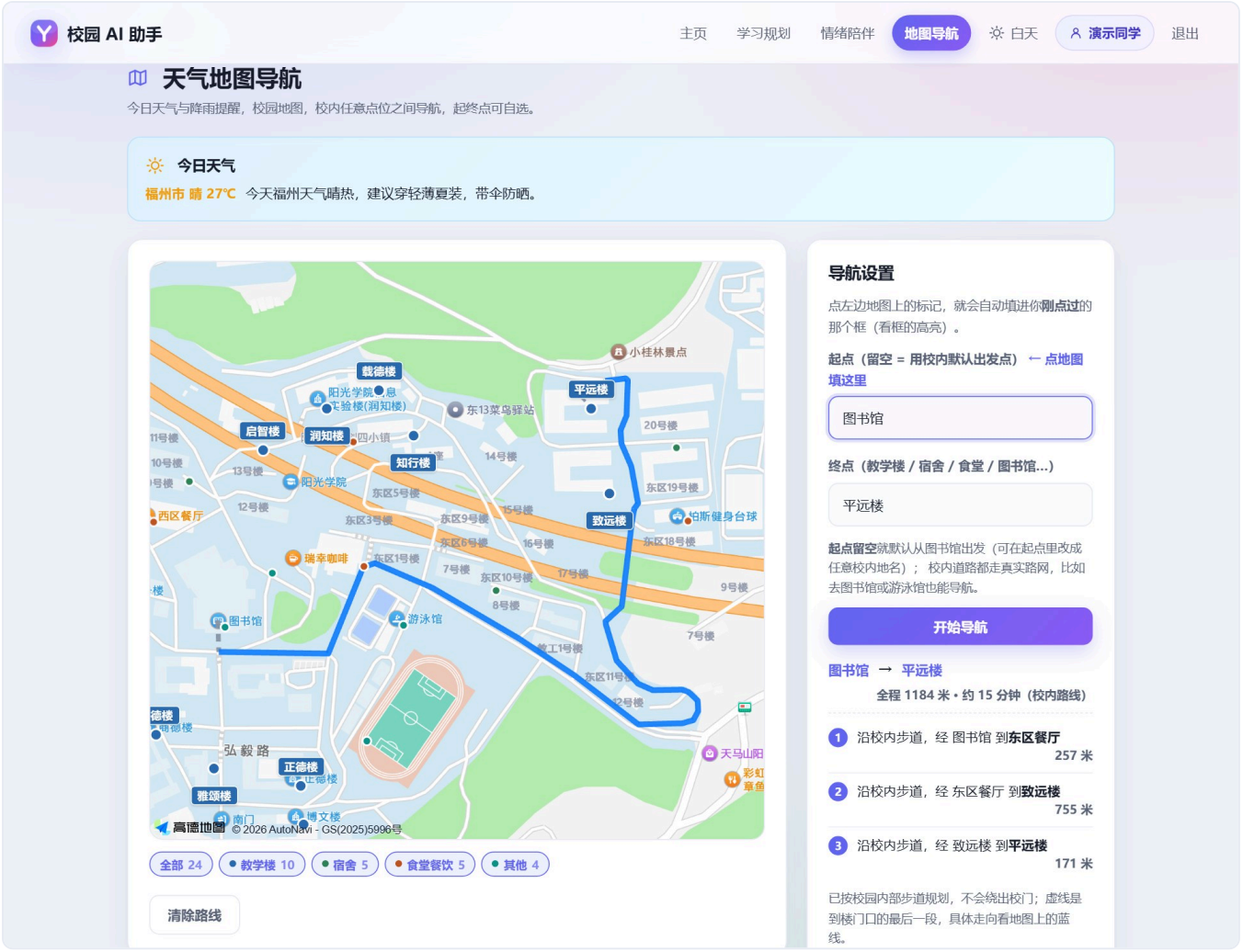


图 5　地图导航：图书馆 → 平远楼 1,184 米 / 约 15 分钟，路线画在底图上，右侧是分段指引

5.5 情绪陪伴与测评

对话：每条消息先取相关记忆（Qdrant `emotion_memory`，embedding-3 2048 维 / embedding-2 1024 维，按实际模型自动定维度），再连同用户消息一起交给 GLM。记忆只用于"接住上次的情绪"，不用于推断结论；清个人数据时整集合删除（与文档语料互不影响）。

测评：8 题 Likert 4 级（完全没有 / 有几天 / 一半以上 / 几乎每天），4 题抑郁维度 + 4 题焦虑维度。**分数由后端计算**，前端只负责展示与分级文案：

- 前端算分等于把"尺子"交给不可信的客户端；
- 分数直接决定分级建议（如"建议联系辅导员"），这类判断不能有第二个来源。

边界：量表是筛查工具，不是诊断工具。文案里明确写"不构成医学诊断"，严重分级时指向校心理中心与辅导员，并给出联系方式入口。

5.6 管理端与反馈闭环

管理员登录后进入独立的「数据」视角（顶栏只有"数据"与"反馈"两个入口）：

页面	内容
`admin.html`	注册用户、近 7 日活跃、课表填写率、情绪对话量、测评分布、反馈类型分布（ECharts 看板）
`admin-feedback.html`	反馈列表，按状态（待处理 / 已处理）筛选
`feedback.html`	学生提交反馈：类型 + 内容，可查看自己的历史

为什么要做这个闭环：如果只有学生端，产品就永远停在"猜用户需要什么"。反馈表 + 看板让"猜"变成"看数据"：哪一类问题被问得最多、哪个页面被反馈最多，下一轮就改哪里。

AI 成本也要看得见：`services/usage.py` 在每次模型调用后记录 `usage`（prompt/completion token），按请求累计、随取随清，避免把上一个用户的用量算到下一个人头上。管理端因此能报出"平均一次对话多少 token"，而不是估算值。

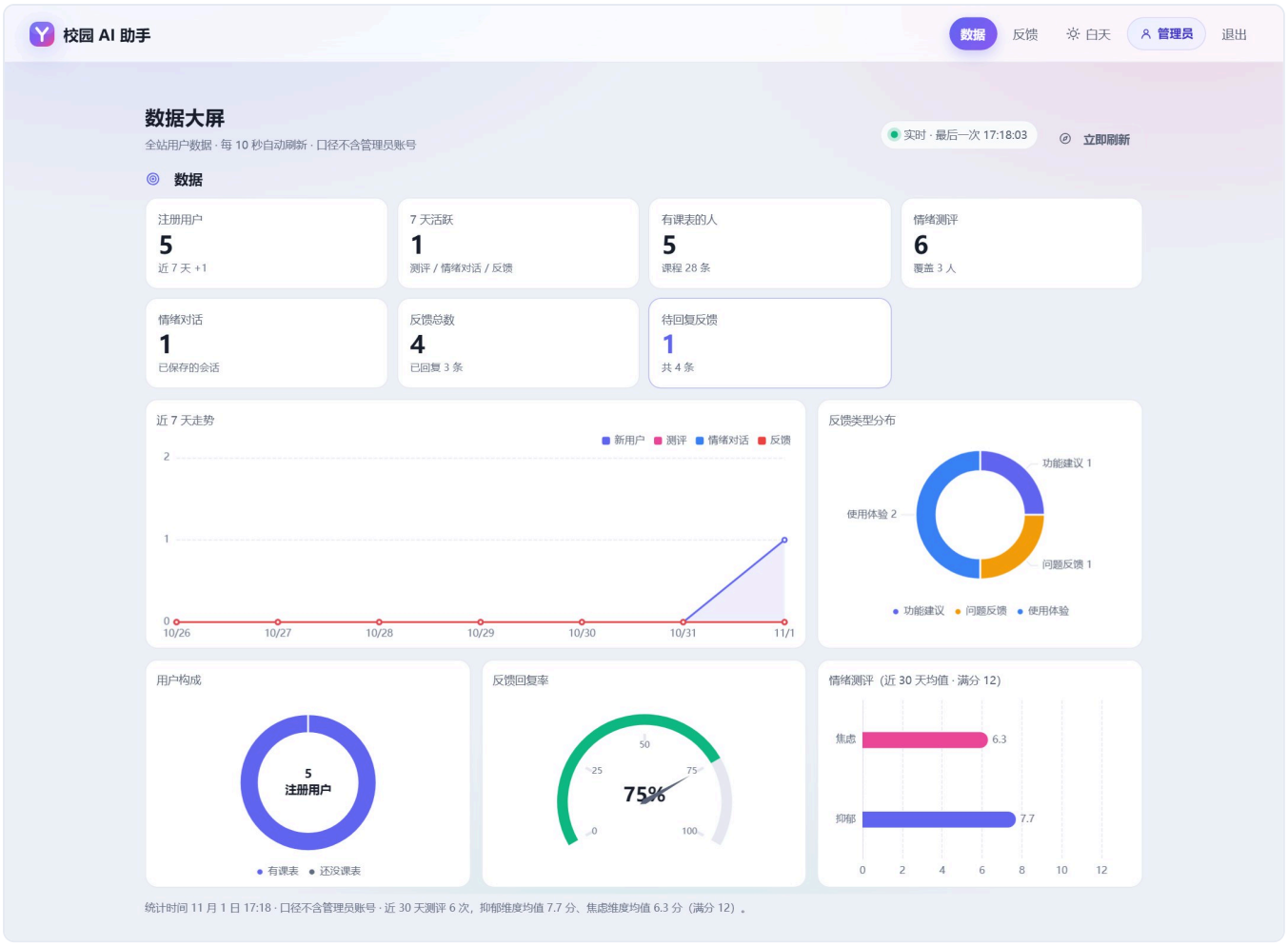


图 6　管理端数据大屏：7 张指标卡 + 4 张图表，每 10 秒自动刷新，口径不含管理员账号

5.7 账号与登录

- 账号密码登录（JWT，`Authorization: Bearer`）；
- 扫码登录：桌面端生成二维码（`qrcode.js` 本地生成，不依赖第三方图片服务），手机扫码后确认，桌面端轮询状态；
- **托管网关会改写** `Authorization`，所以前端同时发 `X-Campus-Token`，后端优先读它、回退读 Bearer。只发一个时线上会 401 死循环，而本机测不出来。

六、测试与部署

6.1 四层测试体系

层	内容	特点
后端契约	Agent 问答契约、RAG 文档、扫码登录、数据导出、Excel 导入	脚本式，自带"结论: N 通过 / M 失败"与退出码
前端纯逻辑	按周判断、周次显示、课表冻结、点位数据	不起服务、不开浏览器，秒级完成
端到端冒烟	逐页打开，断言无控制台错误、无白屏	真浏览器
视觉审计	axe-core 扫 WCAG 2.1 A/AA；多档宽度查横向溢出	浅色 + 黑夜两轮

所有脚本都是**纯 Node / 纯 Python**，没有测试框架依赖，`node tools/xxx.mjs` 直接跑。这样做的原因很实际：评审或队友拿到源码，不需要先学一套 fixture 约定。

6.2 四个来自真实事故的守卫

1. **pandas 缺失**：把 pandas 屏蔽后跑解析器，断言输出与有 pandas 时逐字节相同。理由见 5.1。
2. **课表冻结**：改顶栏或 CSS 后，断言表头行位置不变。曾有一次视觉改动让 sticky 失效，星期行跟着滚走。
3. **两处同口径**：前端的按周判断与后端的 `_course_on_date` 必须对同一组输入给出同一结果。任何一边单独改，都会让"这周有课吗"和课表页显示不一致。
4. **动作词表只有一份**：RAG 的动作词正则被多处共用，守卫断言它只出现一次——写过两张表的那次，"请病假"被绕过、答成了退学流程。

6.3 持续集成

仓库内配置 GitHub Actions（`.github/workflows/ci.yml`）：后端契约 + 覆盖率、前端纯逻辑守卫、端到端 + 无障碍、镜像构建与冒烟。每次推送自动跑；被新提交取消的旧轮次标 cancelled，不算失败。

6.4 部署

本地（评审最省事）：解压 → 双击 `start-windows.bat` → 浏览器自动打开。脚本会自动检测 Python、安装缺失依赖、挑局域网 IP、做一次自测。

容器：包内附 `Dockerfile`，`docker build -t campus-ai . && docker run -p 3000:3000 campus-ai`。

服务器：

```
pip install -r backend/requirements.txt
cd backend && PORT=${PORT:-3000} python main.py
```

- 必须显式传 `PORT`：部分托管平台不注入该变量，缺省会起在 8000，结果是平台健康检查失败（`SITES_START_FAILED`）；
- 前端由后端挂载在 `/app`，不需要额外的静态服务器；
- 首次启动会建库、建时间槽、建评审账号。

6.5 零 Key 可用性矩阵

场景	无 GLM	无 Qdrant	无高德
登录 / 时间表 / 导入	✅	✅	✅
AI 排计划	⚠️ 本地规则计划	✅	✅
手册问答	⚠️ 无法作答，提示未配置	⚠️ 答"手册没查到"	✅
情绪对话	⚠️ 固定关怀话术	⚠️ 无记忆，对话正常	✅
测评	✅	✅	✅
地图导航	✅	✅	⚠️ 底图与天气不可用，路网数据仍在

七、总结与展望

7.1 完成了什么

- 一个能在评审机器上**双击就跑**的完整应用：14 个页面、6 个只读工具、29 篇语料的问答、172 节点路网、8 题量表；
- 一件"别人做不了"的东西：自建校园步道路网，把 1,675 米的绕路变成 680 米，并且**质量可以量化核验**（离路 p90 3.27 米）；
- 一套把 AI 关进笼子的工程约束：只读工具、日期锚点、能算的不调模型、三道反幻觉闸、全程可降级；
- 一条能复现的测试链：18 个脚本、四层体系、仓库内 CI；
- 一条真实的数据闭环：反馈 → 看板 → 下一轮改哪里。

7.2 局限

- **语料只覆盖 29 篇**：覆盖不到的问题只能答"没写"，这不是缺陷而是设计——但用户会觉得"它不会"；
- **校区数据是单校区的**：路网与点位按阳光学院福州校本部构建，换学校要重跑整条管线；
- **记忆与检索依赖外部向量库**：没有 Qdrant 时，情绪记忆退化成本次对话，手册问答完全不可用；
- **并发未做压测**：SQLite + 单进程 uvicorn，适合教学演示规模，不适合真实院系级并发；

- **管理端只做只读看板**：不做用户管理、内容审核。

7.3 展望

1. **语料工程化**：把抓取 → 清洗 → 切块 → 入库做成可配置管线，扩到更多学校与部门；
2. **多校区路网**：把建图参数化，换学校只换底图与标定；
3. **答案可复核**：把"检索片段 + 闸门判定过程"做成可展开的证据面板，方便辅导员核对；
4. **端侧性能**：从 `watchPosition` 的高频回调做起，降低手机端耗电。

八、创新点说明

8.1 技术创新点

T1. 用"底图上真实画出来的白色道路"作为唯一几何来源，自建校园步道图

不掺手画直线、不混第三方路网。代价是建图管线长（截取 → 分割 → 骨架化 → 矢量化 → 建图 → 补桥），收益是**每一条路线都能被核验**：节点离路 p90 3.27 米，离路超 3 米的里程只占 5.7%。同类作品通常直接调地图 API，无法回答"你这路线为什么合理"。

T2. 把反幻觉做成确定性判据，而不是提示词

"片段里没有的不要写"这类提示词约束，模型可以绕。本项目的做法是**算出来**：问句里的动作词在片段中缺了哪几个、检索对象是否张冠李戴，都是可枚举、可断言的。三道闸让"不知道"变成一个明确、可信的答案。

T3. 课表按「周次」建模，不按日期铺开

课程只存「星期几 + 生效周次区间」，某天有没有课由「星期相同 且 该天周次落在区间内」实时判断。单双周、前后八周、临时调课都不需要改数据，换学期只改一个开学日常量。把课程按日期展开成几十条记录的做法，一旦导入或调课就会产生重复行与错位。

T4. 把"AI 不该做的事"写进代码

周次、空档、里程、分数、动作词比对全部本地计算，模型只负责听懂与组织语言。这不是"优化"，而是**正确性前提**：模型算错第几周不会报错，只会给出一份看起来完美的错计划。

8.2 应用创新点

A1. 首屏回答的是"现在要出门吗"，不是"你想聊什么"

依据 53 份问卷：94.3% 出门前要确认几点上课在哪栋楼、81.1% 要算走过去多久、71.7% 要看天气。所以首页放"课表 + 走过去多久 + 天气"，AI 作为补充入口而非主入口。这与"先做聊天框"的常见路径相反。

A2. 过程可见 + 出处可点

Agent 的工具轨迹渲染成时间线，手册答案带原文链接。前者让"依据"看得见，后者让"不编"可核对。两个设计同时是产品功能与工程约束。

A3. 反馈闭环：把"猜需求"变成"看数据"

学生提交反馈 → 管理端看板按类型聚合 → 下一轮迭代挑被反馈最多的地方改。配合 token 用量统计，"这个功能值不值得留"有数据可依。

附录 A　环境配置与依赖

A.1 环境要求

项	要求
操作系统	Windows 10/11（一键脚本）；Linux / macOS / Docker 可手动启动
Python	3.10 及以上
浏览器	Chrome / Edge 现代版本（View Transition 需 Chrome 126+，其余功能不依赖）
内存	512 MB 以上
网络	可选。无网络时除地图底图与 AI 外均可用

A.2 环境变量

完整模板见包内 `backend/.env.example`，复制为 `backend/.env` 后按需填写：

变量	必填	作用	缺失时
`GLM_API_KEY`	否	学业规划、情绪对话、文档问答	降级：本地规则计划 / 固定话术
`GLM_MODEL`	否	对话模型名，默认 `glm-4-flash`	—
`QDRANT_URL` / `QDRANT_API_KEY`	否	文档检索与情绪长期记忆	降级：无记忆、文档问答不可用
`AMAP_KEY` / `AMAP_WEB_KEY`	否	地图与天气	降级：底图与天气不可用
`DATABASE_URL`	否	默认 SQLite	—
`JWT_SECRET`	建议	登录令牌签名	使用内置默认值（仅演示可接受）

密钥不出现在公开仓库：`.env` 被 `.gitignore` 排除，仓库只提交 `.env.example`（全部留空）。交付包内的 `.env` 为演示用，不随仓库分发。

A.3 依赖清单

`backend/requirements.txt`（完整注释见文件内）：

```
fastapi
uvicorn[standard]
sqlalchemy
PyJWT
python-multipart
pydantic-settings
httpx
xlrd
openpyxl
qdrant-client        # 可选，缺失只降级检索与记忆
# pandas             # 可选，缺失时解析器走 xlrd/openpyxl 直读
```

前端零依赖：无 `package.json`、无构建步骤，浏览器直接加载。

A.4 启动

```
# 方式一（Windows）
双击 start-windows.bat

# 方式二（任意平台）
python -m pip install -r backend/requirements.txt
cd backend && PORT=3000 python main.py
# 浏览器打开 http://127.0.0.1:3000/app/

# 方式三（容器）
docker build -t campus-ai . && docker run -p 3000:3000 campus-ai
```

附录 B　评审快速指引

想验证什么	怎么做	预期
能不能跑起来	双击 `start-windows.bat`	浏览器自动打开首页，无需填任何 Key
AI 会不会瞎编	「学习规划」问"下周三下午有课吗"	先出工具轨迹，再给结论；课表里没有就说没有
手册问答准不准	问"休学需要准备什么材料"	给流程 + 原文出处；问覆盖不到的问题答"没写"
路线是不是真校内	「地图导航」起点知行楼、终点平远楼	约 680 米，路线压在校内道路上
管理端是否可用	用 `admin` 登录	看到数据看板与反馈列表
数据是否只增不改	看仓库提交历史与 Actions 运行记录	每次推送都有对应流水线
隐私是否过关	检查 `.gitignore` 与包内是否含 `campus_ai.db`	数据库与导出数据均不在仓库/包内

本文档共八章，覆盖项目概述、需求分析、系统架构、AI 技术方案、核心功能实现、测试与部署、总结与展望、创新点说明。